

L1. NET Technologies(.NET & C#)- Intermediate Assessments

Multi-language Translation Microservice

a. Translates text between languages. It supports batch and real-time translation and is used in applications requiring multilingual support for content or communication.

b. APIs are designed to be integrated into an API Gateway for centralized access and routing.

POST /translate – Translate text

GET /languages – List supported languages

POST /translate/batch – Translate multiple texts

c. In micro services following services could be implemented as separate Web-API to implement Web-API-Gateway.

LanguageDetectionService

TranslationEngineService

GlossaryManagementService

Task:

1. Perform W3H analysis on the given Case Study.
2. Use the Case Study and Create WEB-API to implement Code-First Approach.
3. Generate an OpenAPI (Swagger) specification, API Gateway routing config, Next implement microservice architecture.
4. Perform validations in each model using Data Annotations.
5. Use ASP.Net Entity Framework Core for Back-End Communication.
6. Use MySQL as your database.
7. Use ReactJS as Front-End Application Development.
8. Apply style using CSS / Bootstrap.
9. Perform CRUD operations.
10. Maintain Code Quality and Coding Standard.

W3H Analysis

Project Title: Multi-language Translation Microservice	
What?	How?
1. What are the module needed for this project? A: Modules Required: • LanguageDetectionService. • TranslationEngineService. • GlossaryManagementService. • Front-End UI. 2. What are the features of LanguageDetectionService? A: LanguageDetectionService: • Need API testing Capabilities. • Implement API Methods. • Supports CRUD via a front-end. 3. What are the features of TranslationEngineService? A: TranslationEngineService: • Need API testing Capabilities. • Implement API Methods. • Supports CRUD via a front-end. 4. What are the features of GlossaryManagementService? A: GlossaryManagementService: • Need API testing Capabilities. • Implement API Methods. • Supports CRUD via a front-end.	1. LanguageDetectionService: • Need API testing Capabilities: ◦ Method 1: Use Swagger UI. ◦ Method 2: Use Postman. ◦ Method 3: Use Manual API calls via CLI. • Implement API Methods: ◦ Method 1: Use Controllers only. ◦ Method 2: Use Repositories. ◦ Method 3: Use Service + Repositories. • Supports CRUD via a front-end: ◦ Method 1: Use HTTP. ◦ Method 2: Use HTTPS. ◦ Method 3: Use a common Gateway on either HTTP or HTTPS. 2. TranslationEngineService: • Need API testing Capabilities: ◦ Method 1: Use Swagger UI. ◦ Method 2: Use Postman. ◦ Method 3: Use Manual API calls via CLI. • Implement API Methods: ◦ Method 1: Use Controllers only. ◦ Method 2: Use Repositories. ◦ Method 3: Use Service +

5. What are the features of the Front-End UI?

A: Front-End UI:

- Supports CRUD operations with the services.
- Have ease of access to identify and perform data actions.

Repositories.

- **Supports CRUD via a front-end:**
 - Method 1: Use HTTP.
 - Method 2: Use HTTPS.
 - **Method 3: Use a common Gateway on either HTTP or HTTPS.**

3. GlossaryManagementService:

- **Need API testing Capabilities:**
 - Method 1: Use Swagger UI.
 - **Method 2: Use Postman.**
 - Method 3: Use Manual API calls via CLI.
- **Implement API Methods:**
 - **Method 1: Use Controllers only.**
 - Method 2: Use Repositories.
 - Method 3: Use Service + Repositories.
- **Supports CRUD via a front-end:**
 - Method 1: Use HTTP.
 - Method 2: Use HTTPS.
 - **Method 3: Use a common Gateway on either HTTP or HTTPS.**

4. Front-End UI:

- **Supports CRUD operations with the services:**
 - **Method 1: Use simple forms.**
 - Method 2: Use JSON data as input.
 - Method 3: Simply redirect to Postman.
- **Have ease of access to identify and perform data actions:**
 - **Method 1: Colour Code the**

	buttons. - Method 2: Hovering with always show detailed explanation of the button. - Method 3: Add accessibility tools.
1. LanguageDetectionService: Need API testing Capabilities: Method 2: Use Postman. A: Highly efficient with proper UI support to test and document APIs. Implement API Methods: Method 1: Use Controllers only. A: Simple and efficient implementation methodology for a smaller version of this case. Supports CRUD via a front-end: Method 3: Use a common Gateway on either HTTP or HTTPS. A: Supports multiple APIs at once in an effective manner. 2. TranslationEngineService: Need API testing Capabilities: Method 2: Use Postman. A: Highly efficient with proper UI support to test and document APIs. Implement API Methods: Method 1: Use Controllers only. A: Simple and efficient implementation methodology for a smaller version of	1. LanguageDetectionService: Need API testing Capabilities: - Method 1: Use Swagger UI. - Method 3: Use Manual API calls via CLI. A: In-efficient and may cause errors if middle ware configuration is not don't correctly. Implement API Methods: - Method 2: Use Repositories. - Method 3: Use Service + Repositories. A: Too complex for a small scale prototype. Supports CRUD via a front-end: - Method 1: Use HTTP. - Method 2: Use HTTPS. A: Does the job but due to existence of multiple APIs, this alone cant suffice. 2. TranslationEngineService: Need API testing Capabilities: - Method 1: Use Swagger UI. - Method 3: Use Manual API calls via CLI. A: In-efficient and may cause errors if middle ware configuration is not don't correctly.

this case.

- **Supports CRUD via a front-end:**
 - **Method 3: Use a common Gateway on either HTTP or HTTPS.**

A: Supports multiple APIs at once in an effective manner.

3. GlossaryManagementService:

- **Need API testing Capabilities:**
 - **Method 2: Use Postman.**

A: Highly efficient with proper UI support to test and document APIs.

- **Implement API Methods:**
 - **Method 1: Use Controllers only.**

A: Simple and efficient implementation methodology for a smaller version of this case.

- **Supports CRUD via a front-end:**
 - **Method 3: Use a common Gateway on either HTTP or HTTPS.**

A: Supports multiple APIs at once in an effective manner.

4. Front-End UI:

- **Supports CRUD operations with the services:**
 - **Method 1: Use simple forms.**

A: Efficient and Simple to implement/

- **Have ease of access to identify and perform data actions:**

- **Implement API Methods:**
 - Method 2: Use Repositories.
 - Method 3: Use Service + Repositories.

A: Too complex for a small scale prototype.

- **Supports CRUD via a front-end:**
 - Method 1: Use HTTP.
 - Method 2: Use HTTPS.

A: Does the job but due to existence of multiple APIs, this alone cant suffice.

3. GlossaryManagementService:

- **Need API testing Capabilities:**
 - Method 1: Use Swagger UI.
 - Method 3: Use Manual API calls via CLI.

A: In-efficient and may cause errors if middle ware configuration is not don't correctly.

- **Implement API Methods:**
 - Method 2: Use Repositories.
 - Method 3: Use Service + Repositories.

A: Too complex for a small scale prototype.

- **Supports CRUD via a front-end:**
 - Method 1: Use HTTP.
 - Method 2: Use HTTPS.

A: Does the job but due to existence of multiple APIs, this alone cant suffice.

4. Front-End UI:

- **Supports CRUD operations with the**

◦ Method 1: Colour Code the buttons. A: Easier solution to address the issue.	services: ◦ Method 2: Use JSON data as input. ◦ Method 3: Simply redirect to Postman. A: Highly roundabout methodologies that don't promise efficient results. • Have ease of access to identify and perform data actions: ◦ Method 2: Hovering with always show detailed explanation of the button. ◦ Method 3: Add accessibility tools. A: Highly roundabout methodologies that don't promise efficient results.
Why?	Why Not?

CODE

All APIs Have Common Program.cs

```
//Program.cs

using Microsoft.EntityFrameworkCore;
using TranslationEngineService.Data;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllers();

// Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();

builder.Services.AddDbContext<AppDbContext>(options =>
options.UseMySQL(builder.Configuration.GetConnectionString("DefaultConnection"),

ServerVersion.AutoDetect(builder.Configuration.GetConnectionString("DefaultConnection"
))));

// Add CORS policy
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowLocalhostFrontend", policy =>
    {
        policy.WithOrigins("http://localhost:63399")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});

var app = builder.Build();

app.UseCors("AllowLocalhostFrontend");
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

TranslationEngineService

Model

```
using System.ComponentModel.DataAnnotations;

namespace TranslationEngineService.Model
{
    public class TranslationEngine
    {
        [Key]
        public int Id { get; set; }

        public string TranslationString { get; set; }

        public string FromLang { get; set; }

        public string ToLang { get; set; }
    }
}
```


Data

```
//AppDbContext.cs

using Microsoft.EntityFrameworkCore;
using TranslationEngineService.Model;

namespace TranslationEngineService.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
        { }

        public DbSet<TranslationEngine> TranslationEngines{ get; set; }
    }
}
```

Controller

```
//Controller.cs
```

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using TranslationEngineService.Data;
using TranslationEngineService.Model;

namespace TranslationEngineService.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class TranslationEngineController : ControllerBase
    {
        private readonly AppDbContext _Context;

        public TranslationEngineController(AppDbContext context)
        {
            _Context = context;
        }

        //Get: api/
        [HttpGet]
        public async Task<IActionResult> GetAll()
        {
            var students = await _Context.TranslationEngines.ToListAsync();
            return Ok(students);
        }

        [HttpGet("{id}")]
        public async Task<IActionResult> Get(int id)
        {
            var translationItem = await _Context.TranslationEngines.FindAsync(id);
            if (translationItem == null) return NotFound();
            else return Ok(translationItem);
        }

        [HttpPost]
        public async Task<IActionResult> Create([FromBody] TranslationEngine
translationItem)
        {
            _Context.TranslationEngines.Add(translationItem);
            await _Context.SaveChangesAsync();
            return CreatedAtAction(nameof(Create), translationItem);
        }

        [HttpPut("{id}")]
        public async Task<IActionResult> Update(int id, [FromBody] TranslationEngine
translationItem)
        {
            var translationItemToUpdate = await
_Context.TranslationEngines.FindAsync(id);
            if (translationItemToUpdate == null) return NotFound();
            translationItemToUpdate.TranslationString =
translationItem.TranslationString;
            await _Context.SaveChangesAsync();
            return NoContent();
        }

        [HttpDelete("{id}")]
        public async Task<IActionResult> Delete(int id)
        {

```

```
var student = await _Context.TranslationEngines.FindAsync(id);
if (student == null) return NotFound();
_Context.TranslationEngines.Remove(student);
await _Context.SaveChangesAsync();
return NoContent();
    }
}
}
```

appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "ConnectionStrings": {
    "DefaultConnection":
"Server=127.0.0.1;Port=3306;Database=TranslationEngineServiceDB;User=root;Password=root"
  },
  "AllowedHosts": "*"
}
```

GlossaryManagementService

Model

```
using System.ComponentModel.DataAnnotations;

namespace GlossaryManagementService.Model
{
    public class GlossaryManagement
    {
        [Key]
        public int Id { get; set; }

        public string Word { get; set; }

        public string Meaning { get; set; }

        public string? Notes { get; set; }
    }
}
```

Data

```
//AppDbContext.cs

using Microsoft.EntityFrameworkCore;
using GlossaryManagementService.Model;

namespace GlossaryManagementService.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
        { }

        public DbSet<GlossaryManagement> GlossaryManagements { get; set; }
    }
}
```

Controller

//Controller.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using GlossaryManagementService.Data;
using GlossaryManagementService.Model;

namespace GlossaryManagementService.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class GlossaryManagementController : ControllerBase
    {
        private readonly ApplicationDbContext _Context;

        public GlossaryManagementController(ApplicationDbContext context)
        {
            _Context = context;
        }

        //Get: api/
        [HttpGet]
        public async Task<IActionResult> GetAll()
        {
            var glosssaryItems = await _Context.GlossaryManagements.ToListAsync();
            return Ok(glosssaryItems);
        }

        [HttpGet("{id}")]
        public async Task<IActionResult> Get(int id)
        {
            var glosssaryItem = await _Context.GlossaryManagements.FindAsync(id);
            if (glosssaryItem == null) return NotFound();
            else return Ok(glosssaryItem);
        }

        [HttpPost]
        public async Task<IActionResult> Create([FromBody] GlossaryManagement
glosssaryItem)
        {
            _Context.GlossaryManagements.Add(glosssaryItem);
            await _Context.SaveChangesAsync();
            return CreatedAtAction(nameof(Create), glosssaryItem);
        }

        [HttpPut("{id}")]
        public async Task<IActionResult> Update(int id, [FromBody] GlossaryManagement
glosssaryItem)
        {
            var glosssaryItemToUpdate = await
_Context.GlossaryManagements.FindAsync(id);
            if (glosssaryItemToUpdate == null) return NotFound();
            glosssaryItemToUpdate.Word = glosssaryItem.Word;
            glosssaryItemToUpdate.Meaning = glosssaryItem.Meaning;
            glosssaryItemToUpdate.Notes = glosssaryItem.Notes;
            await _Context.SaveChangesAsync();
            return NoContent();
        }

        [HttpDelete("{id}")]
        public async Task<IActionResult> Delete(int id)
```

```
    {
        var glosssaryItem = await _Context.GlossaryManagements.FindAsync(id);
        if (glosssaryItem == null) return NotFound();
        _Context.GlossaryManagements.Remove(glosssaryItem);
        await _Context.SaveChangesAsync();
        return NoContent();
    }
}
```


appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "ConnectionStrings": {
    "DefaultConnection":
"Server=127.0.0.1;Port=3306;Database=GlossaryManagementServiceDB;User=root;Password=root"
  },
  "AllowedHosts": "*"
}
```

LanguageDetectionService

Model

```
using System.ComponentModel.DataAnnotations;

namespace LanguageDetectionService.Model
{
    public class LanguageDetection
    {
        [Key]
        public int Id { get; set; }

        public string Name { get; set; }

        public string? Origin { get; set; }

        public string? Notes { get; set; }
    }
}
```

Data

```
//AppDbContext.cs

using Microsoft.EntityFrameworkCore;
using LanguageDetectionService.Model;

namespace LanguageDetectionService.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
        { }

        public DbSet<LanguageDetection> LanguageDetections { get; set; }
    }
}
```

Controller

```
//Controller.cs
```

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using LanguageDetectionService.Data;
using LanguageDetectionService.Model;

namespace LanguageDetectionService.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class LanguageDetectionController : ControllerBase
    {
        private readonly ApplicationDbContext _Context;

        public LanguageDetectionController(ApplicationDbContext context)
        {
            _Context = context;
        }

        //Get: api/
        [HttpGet]
        public async Task<IActionResult> GetAll()
        {
            var languageItems = await _Context.LanguageDetections.ToListAsync();
            return Ok(languageItems);
        }

        [HttpGet("{id}")]
        public async Task<IActionResult> Get(int id)
        {
            var languageItem = await _Context.LanguageDetections.FindAsync(id);
            if (languageItem == null) return NotFound();
            else return Ok(languageItem);
        }

        [HttpPost]
        public async Task<IActionResult> Create([FromBody] LanguageDetection
languageItem)
        {
            _Context.LanguageDetections.Add(languageItem);
            await _Context.SaveChangesAsync();
            return CreatedAtAction(nameof(Create), languageItem);
        }

        [HttpPut("{id}")]
        public async Task<IActionResult> Update(int id, [FromBody] LanguageDetection
languageItem)
        {
            var languageItemToUpdate = await
_Context.LanguageDetections.FindAsync(id);
            if (languageItemToUpdate == null) return NotFound();
            languageItemToUpdate.Name = languageItem.Name;
            languageItemToUpdate.Origin = languageItem.Origin;
            languageItemToUpdate.Notes = languageItem.Notes;
            await _Context.SaveChangesAsync();
            return NoContent();
        }

        [HttpDelete("{id}")]
        public async Task<IActionResult> Delete(int id)
```

```
    {
        var student = await _Context.LanguageDetections.FindAsync(id);
        if (student == null) return NotFound();
        _Context.LanguageDetections.Remove(student);
        await _Context.SaveChangesAsync();
        return NoContent();
    }
}
```

appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "ConnectionStrings": {
    "DefaultConnection":
"Server=127.0.0.1;Port=3306;Database=LanguageDetectionServiceDB;User=root;Password=root"
  },
  "AllowedHosts": "*"
}
```

API Gateway

Program.cs

```
//Program.cs - Ocelot

using Ocelot.DependencyInjection;
using Ocelot.Middleware;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Configuration.AddJsonFile("ocelot.json", optional: false, reloadOnChange:
true);
builder.Services.AddOcelot();
builder.Services.AddControllers();

// Add CORS policy
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowLocalhostFrontend", policy =>
    {
        policy.WithOrigins("http://localhost:63399")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});

var app = builder.Build();

app.UseCors("AllowLocalhostFrontend");

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
await app.UseOcelot();
app.Run();
```

Ocelot.json

```
//Ocelot.json
```

```
{
  "Routes": [
    {
      "UpstreamPathTemplate": "/gateway/GlossaryManagement",
      "UpstreamHttpMethod": [ "GET", "POST" ],
      "DownstreamPathTemplate": "/api/GlossaryManagement",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5276
        }
      ]
    },
    {
      "UpstreamPathTemplate": "/gateway/GlossaryManagement/{id}",
      "UpstreamHttpMethod": [ "GET", "PUT", "DELETE" ],
      "DownstreamPathTemplate": "/api/GlossaryManagement/{id}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5276
        }
      ]
    },
    {
      "UpstreamPathTemplate": "/gateway/LanguageDetection",
      "UpstreamHttpMethod": [ "GET", "POST" ],
      "DownstreamPathTemplate": "/api/LanguageDetection",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5194
        }
      ]
    },
    {
      "UpstreamPathTemplate": "/gateway/LanguageDetection/{id}",
      "UpstreamHttpMethod": [ "GET", "PUT", "DELETE" ],
      "DownstreamPathTemplate": "/api/LanguageDetection/{id}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5194
        }
      ]
    },
    {
      "UpstreamPathTemplate": "/gateway/TranslationEngine",
      "UpstreamHttpMethod": [ "GET", "POST" ],
      "DownstreamPathTemplate": "/api/TranslationEngine",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5233
        }
      ]
    }
  ]
}
```



```
    }  
  ]  
},  
{  
  "UpstreamPathTemplate": "/gateway/TranslationEngine/{id}",  
  "UpstreamHttpMethod": [ "GET", "PUT", "DELETE" ],  
  "DownstreamPathTemplate": "/api/TranslationEngine/{id}",  
  "DownstreamScheme": "http",  
  "DownstreamHostAndPorts": [  
    {  
      "Host": "localhost",  
      "Port": 5233  
    }  
  ]  
}  
]  
}
```

React Front-End Application

Components

//GlossaryForm.jsx

```
import React, { useState, useEffect } from 'react';

const GlossaryForm = ({ onSubmit, initial }) => {
  const [form, setForm] = useState(initial);

  useEffect(() => {
    setForm(initial);
  }, [initial]);

  const handleSubmit = (e) => {
    e.preventDefault();

    if (form.notes == null || form.notes == "") {
      form.notes = "No Notes."
    }

    onSubmit(form);
    setForm({ word: '', meaning: '', notes: '' });
  };

  return (
    <form onSubmit={handleSubmit}>
      <table>
        <tbody>
          <tr>
            <td>
              <input
                type="text"
                placeholder="Enter the word."
                value={form.word || ''}
                onChange={(e) => setForm({ ...form, word:
e.target.value }}}
                required
              />
            </td>
          </tr>
          <tr>
            <td>
              <input
                type="text"
                placeholder="Enter the meaning."
                value={form.meaning || ''}
                onChange={(e) => setForm({ ...form, meaning:
e.target.value }}}
                required
              />
            </td>
          </tr>
          <tr>
            <td>
              <input
                type="text"
                placeholder="Any additional notes?"
```

```

        value={form.notes || ''}
        onChange={e => setForm({ ...form, notes:
e.target.value })}}
      />
    </td>
  </tr>
  <tr>
    <td>
      <button type="submit">{initial.id ? 'Update' : 'Add'}
Glossary</button>
    </td>
  </tr>
</tbody>
</table>
</form>
  );
};

export default GlossaryForm;

```

```

//GlossaryList.jsx

import React, { useState } from 'react';

const GlossaryList = ({ glossaries, onEdit, onDelete }) => {
  const [searchTerm, setSearchTerm] = useState('');

  const handleSearch = () => {
    const results = glossaries.filter(g =>
      g.word.toLowerCase().includes(searchTerm.toLowerCase())
    );
    const message = results.length
      ? 'Search Results:\n' + results.map(g => `Word: ${g.word} \nMeaning: ${
        {g.meaning} \nAdditional Notes: ${g.notes}}`).join('\n')
      : 'No matching glossaries found.';
    alert(message);
  };

  if (!glossaries || glossaries.length === 0) {
    return <p>No glossaries found.</p>;
  }

  return (
    <>
      <h2>Glossary Search:</h2>
      <div style={{ marginBottom: '40px', display: "flex" }}>
        <input
          type="text"
          placeholder="Enter the word."
          value={searchTerm}
          onChange={(e) => setSearchTerm(e.target.value)}
        />
        <button onClick={handleSearch}>Search</button>
      </div>

      <h2>Glossary List</h2>
      <table className="list" style={{ borderCollapse: 'collapse', width:
'100%', marginBottom: '80px' }}>
        <thead>
          <tr>
            <th>Word</th>
            <th>Meaning</th>
            <th>Additional Notes</th>
            <th>Edit</th>
            <th>Delete</th>
          </tr>
        </thead>
        <tbody>
          {glossaries.map((g) => (
            <tr key={g.id}>
              <td>{g.word}</td>
              <td>{g.meaning}</td>
              <td>{g.notes}</td>
              <td>
                <button onClick={() => onEdit(g)}>Edit</button>
              </td>
              <td>
                <button onClick={() => onDelete(g.id)}>Delete</button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </>
  );
};

```

```
};
```

```
export default GlossaryList;
```

```
//LanguageForm.jsx
```

```
import React, { useState, useEffect } from 'react';
```

```
const LanguageForm = ({ onSubmit, initial }) => {  
  const [form, setForm] = useState(initial);
```

```
  useEffect(() => {  
    setForm(initial);  
  }, [initial]);
```

```
  const handleSubmit = (e) => {  
    e.preventDefault();
```

```
    if (form.origin == null || form.origin == "") {  
      form.origin = "Not Given."  
    }
```

```
    if (form.notes == null || form.notes == "") {  
      form.notes = "No Notes."  
    }
```

```
    onSubmit(form);  
    setForm({ name: '', origin: '', notes: '' });  
  };
```

```
  return (
```

```
    <form onSubmit={handleSubmit}>
```

```
      <table>
```

```
        <tbody>
```

```
          <tr>
```

```
            <td>
```

```
              <input
```

```
                type="text"
```

```
                placeholder="Enter the name."
```

```
                value={form.name || ''}
```

```
                onChange={(e) => setForm({ ...form, name:
```

```
e.target.value }}}
```

```
                required
```

```
              />
```

```
            </td>
```

```
          </tr>
```

```
          <tr>
```

```
            <td>
```

```
              <input
```

```
                type="text"
```

```
                placeholder="Enter the origin."
```

```
                value={form.origin || ''}
```

```
                onChange={(e) => setForm({ ...form, origin:
```

```
e.target.value }}}
```

```
              />
```

```
            </td>
```

```
          </tr>
```

```
          <tr>
```

```
            <td>
```

```
              <input
```

```
                type="text"
```

```
                placeholder="Any additional notes?"
```

```
                value={form.notes || ''}
```

```
                onChange={(e) => setForm({ ...form, notes:
```

```
e.target.value }}}
```

```
              />
```

```
            </td>
```

```
          </tr>
```

```

        <tr>
          <td>
            <button type="submit">{initial.id ? 'Update' : 'Add'}
Language</button>
          </td>
        </tr>
      </tbody>
    </table>
  </form>
);
};
export default LanguageForm;
```

```

//LanguageList.jsx

import React, { useState } from 'react';

const LanguageList = ({ languages, onEdit, onDelete }) => {
  const [searchTerm, setSearchTerm] = useState('');

  const handleSearch = () => {
    const results = languages.filter(l =>
      l.name.toLowerCase().includes(searchTerm.toLowerCase())
    );
    const message = results.length
      ? 'Search Results:\n' + results.map(l => `Name: ${l.name} \nOrigin: ${
        l.origin} \nAdditional Notes: ${l.notes}`).join('\n')
      : 'No matching languages found.';
    alert(message);
  };

  if (!languages || languages.length === 0) {
    return <p>No languages found.</p>;
  }

  return (
    <>
      <h2>Language Search:</h2>
      <div style={{ marginBottom: '40px', display: "flex" }}>
        <input
          type="text"
          placeholder="Enter the language name."
          value={searchTerm}
          onChange={(e) => setSearchTerm(e.target.value)}
        />
        <button onClick={handleSearch}>Search</button>
      </div>

      <h2>Language List</h2>
      <table className="list" style={{ borderCollapse: 'collapse', width:
'100%', marginBottom: '80px' }}>
        <thead>
          <tr>
            <th>Name</th>
            <th>Origin</th>
            <th>Additional Notes</th>
            <th>Edit</th>
            <th>Delete</th>
          </tr>
        </thead>
        <tbody>
          {languages.map((l) => (
            <tr key={l.id}>
              <td>{l.name}</td>
              <td>{l.origin}</td>
              <td>{l.notes}</td>
              <td>
                <button onClick={() => onEdit(l)}>Edit</button>
              </td>
              <td>
                <button onClick={() => onDelete(l.id)}>Delete</button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </>
  );
};

```



```
};
```

```
export default LanguageList;
```

```

//TranslationForm.jsx

import React, { useState, useEffect } from 'react';

const TranslationForm = ({ onSubmit, initial }) => {
  const [form, setForm] = useState(initial);

  useEffect(() => {
    setForm(initial);
  }, [initial]);

  const handleSubmit = (e) => {
    e.preventDefault();
    onSubmit(form);
    setForm({ translationString: '', fromLang: '', toLang: '' });
  };

  return (
    <form onSubmit={handleSubmit}>
      <table>
        <tbody>
          <tr>
            <td>
              <input
                type="text"
                placeholder="Enter the translation string."
                value={form.translationString || ''}
                onChange={(e) => setForm({ ...form, translationString:
e.target.value }}}
                required
              />
            </td>
          </tr>
          <tr>
            <td>
              <input
                type="text"
                placeholder="Translate From..."
                value={form.fromLang || ''}
                onChange={(e) => setForm({ ...form, fromLang:
e.target.value }}}
                required
              />
            </td>
          </tr>
          <tr>
            <td>
              <input
                type="text"
                placeholder="Translate To..."
                value={form.toLang || ''}
                onChange={(e) => setForm({ ...form, toLang:
e.target.value }}}
                required
              />
            </td>
          </tr>
          <tr>
            <td>
              <button type="submit">{initial.id ? 'Update' : 'Add'}
Translation</button>
            </td>
          </tr>
        </tbody>
      </table>
    </form>
  );
};

```

```
        </table>
      </form>
    );
};

export default TranslationForm;
```

```

//TranslationList.jsx

import React, { useState } from 'react';

const TranslationList = ({ translations, onEdit, onDelete }) => {
  const [searchTerm, setSearchTerm] = useState('');

  const handleSearch = () => {
    const results = translations.filter(t =>
      t.translationString.toLowerCase().includes(searchTerm.toLowerCase())
    );
    const message = results.length
      ? 'Search Results:\n' + results.map(t => `Translation String: $
{t.translationString} \nTranslate From: ${t.fromLang} \nTranslate To: $
{t.toLang}`).join('\n')
      : 'No matching translations found.';
    alert(message);
  };

  if (!translations || translations.length === 0) {
    return <p>No translations found.</p>;
  }

  return (
    <>
      <h2>Translation Search:</h2>
      <div style={{ marginBottom: '40px', display: 'flex' }}>
        <input
          type="text"
          placeholder="Enter the translation string."
          value={searchTerm}
          onChange={(e) => setSearchTerm(e.target.value)}
        />
        <button onClick={handleSearch}>Search</button>
      </div>

      <h2>Translation List</h2>
      <table className="list" style={{ borderCollapse: 'collapse', width:
'100%', marginBottom: '80px' }}>
        <thead>
          <tr>
            <th>Translation String</th>
            <th>Translate From</th>
            <th>Translate To</th>
            <th>Edit</th>
            <th>Delete</th>
          </tr>
        </thead>
        <tbody>
          {translations.map((t) => (
            <tr key={t.id}>
              <td>{t.translationString}</td>
              <td>{t.fromLang}</td>
              <td>{t.toLang}</td>
              <td>
                <button onClick={() => onEdit(t)}>Edit</button>
              </td>
              <td>
                <button onClick={() => onDelete(t.id)}>Delete</button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </>
  );
}

```

```
    );  
};
```

```
export default TranslationList;
```

Pages

Home Page

```
import React, { useEffect } from 'react';
import '../App.css'

function HomePage() {

  useEffect(() => {

    document.title = "Translator-Inator";

  }, []);

  return (
    <div className="container">
      <h1>Welcome to Translator-Inator!</h1>
      <h4>An obvious "Phineas and Ferb" Reference!</h4>

      <br />

      <p>Lorem ipsum dolor sit amet consectetur adipiscing elit. Quisque faucibus
ex sapien vitae pellentesque sem placerat. In id cursus mi pretium tellus dui
convallis. Tempus leo eu aenean sed diam urna tempor. Pulvinar vivamus fringilla lacus
nec metus bibendum egestas. Iaculis massa nisl malesuada lacinia integer nunc posuere.
Ut hendrerit semper vel class aptent taciti sociosqu. Ad litora torquent per conubia
nostra inceptos himenaeos.</p>
    </div>
  );
}

export default HomePage;
```

GlossaryPage

```
//GlossaryPage.jsx
```

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';
import GlossaryForm from '../components/GlossaryForm';
import GlossaryList from '../components/GlossaryList';
import '../App.css'
const GlossaryAPI_BASE = 'http://localhost:5082/gateway/GlossaryManagement';

function GlossaryPage() {
  const [glossaries, setglossaries] = useState([]);
  const [editing, setEditing] = useState(null);
  const [version, setVersion] = useState(0);

  useEffect(() => {
    fetchglossaries();

    document.title = "Glossary Manager";

  }, [version]);

  const fetchglossaries = () => {
    axios.get(GlossaryAPI_BASE).then(res => setglossaries(res.data));
  };

  const addGlossary = (Glossary) => {
    axios.post(GlossaryAPI_BASE, Glossary).then(res => {
      setglossaries([...glossaries, res.data]);
    });
  };

  const updateGlossary = (id, updated) => {
    axios.put(`${GlossaryAPI_BASE}/${id}`, updated).then(res => {
      setglossaries(glossaries.map(p => (p.id === id ? res.data : p)));
      setVersion(prev => prev + 1);
      setEditing(null);
    });
  };

  const deleteGlossary = (id) => {
    axios.delete(`${GlossaryAPI_BASE}/${id}`).then(() => {
      setglossaries(glossaries.filter(p => p.id !== id));
    });
  };

  return (
    <div className="container">
      <div style={{ marginBottom: '40px' }} >
        <h2>Glossary Manager</h2>
        <GlossaryForm
          onSubmit={editing ? (data) => updateGlossary(editing.id, data) :
addGlossary}
          initial={editing || { word: '', meaning: '', notes: '' }}
        />
      </div>
      <div>
        <GlossaryList
          key={version}
          glossaries={glossaries}
          onEdit={setEditing}
          onDelete={deleteGlossary}
        />
      </div>
    </div>
  );
}
```

```
        />
      </div>
    </div>
  );
}

export default GlossaryPage;
```


LanguagePage

//LanguagePage.jsx

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';
import LanguageForm from '../components/LanguageForm';
import LanguageList from '../components/LanguageList';
import '../App.css'
const LanguageAPI_BASE = 'http://localhost:5082/gateway/LanguageDetection';

function LanguagePage() {
  const [languages, setlanguages] = useState([]);
  const [editing, setEditing] = useState(null);
  const [version, setVersion] = useState(0);

  useEffect(() => {
    fetchlanguages();

    document.title = "Language Manager";

  }, [version]);

  const fetchlanguages = () => {
    axios.get(LanguageAPI_BASE).then(res => setlanguages(res.data));
  };

  const addLanguage = (Language) => {
    axios.post(LanguageAPI_BASE, Language).then(res => {
      setlanguages([...languages, res.data]);
    });
  };

  const updateLanguage = (id, updated) => {
    axios.put(`${LanguageAPI_BASE}/${id}`, updated).then(res => {
      setlanguages(languages.map(p => (p.id === id ? res.data : p)));
      setVersion(prev => prev + 1);
      setEditing(null);
    });
  };

  const deleteLanguage = (id) => {
    axios.delete(`${LanguageAPI_BASE}/${id}`).then(() => {
      setlanguages(languages.filter(p => p.id !== id));
    });
  };

  return (
    <div className="container">
      <div style={{ marginBottom: '40px' }} >
        <h2>Language Manager</h2>
        <LanguageForm
          addLanguage={addLanguage}
          initial={editing || { name: '', origin: '', notes: '' }}
        />
      </div>
      <div>
        <LanguageList
          key={version}
          languages={languages}
          onEdit={setEditing}
          onDelete={deleteLanguage}
        />
      </div>
    </div>
  );
}
```

```
        />
      </div>
    </div>
  );
}

export default LanguagePage;
```

TranslationPage

```
//TranslationPage.jsx

import React, { useEffect, useState } from 'react';
import axios from 'axios';
import TranslationForm from '../components/TranslationForm';
import TranslationList from '../components/TranslationList';
import '../App.css'
const TranslationAPI_BASE = 'http://localhost:5082/gateway/TranslationEngine';

function TranslationPage() {
  const [translations, settranslations] = useState([]);
  const [editing, setEditing] = useState(null);
  const [version, setVersion] = useState(0);

  useEffect(() => {
    fetchtranslations();

    document.title = "Translation Manager";

  }, [version]);

  const fetchtranslations = () => {
    axios.get(TranslationAPI_BASE).then(res => settranslations(res.data));
  };

  const addTranslation = (Translation) => {
    axios.post(TranslationAPI_BASE, Translation).then(res => {
      settranslations([...translations, res.data]);
    });
  };

  const updateTranslation = (id, updated) => {
    axios.put(`${TranslationAPI_BASE}/${id}`, updated).then(res => {
      settranslations(translations.map(p => (p.id === id ? res.data : p)));
      setVersion(prev => prev + 1);
      setEditing(null);
    });
  };

  const deleteTranslation = (id) => {
    axios.delete(`${TranslationAPI_BASE}/${id}`).then(() => {
      settranslations(translations.filter(p => p.id !== id));
    });
  };

  return (
    <div className="container">
      <div style={{ marginBottom: '40px' }} >
        <h2>Translation Manager</h2>
        <TranslationForm
          onSubmit={editing ? (data) => updateTranslation(editing.id, data)
: addTranslation}
          initial={editing || { translationString: '', fromLang: '', toLang:
'' }}
        />
      </div>
      <div>
        <TranslationList
          key={version}
          translations={translations}
          onEdit={setEditing}

```

```
        onDelete={deleteTranslation}
      />
    </div>
  </div>
);
}

export default TranslationPage;
```

App.css

```
/*App.css*/

#root {
  margin: 0 auto;
  text-align: center;
}

input {
  padding: 10px;
  width: 100%;
  border-radius: 5px;
  margin: 5px;
  border: none;
}

.container {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh; /* Full viewport height */
  flex-direction: column;
}

.list {
  padding: 20px;
}

th {
  padding: 10px;
}
```

App.jsx

```
//App.jsx

import {
  Routes,
  Route,
  Link
} from 'react-router-dom';

import GlossaryPage from './pages/GlossaryPage';
import LanguagePage from './pages/LanguagePage';
import TranslationPage from './pages/TranslationPage';
import HomePage from './pages/HomePage';

function App() {
  return (
    <div className="container" style={{ marginTop: '30px' }}>
      <table style={{ width: '100vh', borderCollapse: 'collapse' }}>
        <tbody>
          <tr>
            <td colspan="4">
              <nav style={{ marginBottom: '1rem', width: '100%' }}>
                <Link to="/"><button>Home</button></Link>
                &nbsp; &nbsp; &nbsp;
                <Link
to="/glossaryManager"><button>Glossary</button></Link>
                &nbsp; &nbsp; &nbsp;
                <Link
to="/languageManager"><button>Language</button></Link>
                &nbsp; &nbsp; &nbsp;
                <Link
to="/translationManager"><button>Translation</button></Link>
              </nav>
            </td>
            <td colspan="3">
              <Routes>
                <Route path="/" element={<HomePage />} />
                <Route path="/glossaryManager" element={<GlossaryPage
/>} />
                <Route path="/languageManager" element={<LanguagePage
/>} />
                <Route path="/translationManager"
element={<TranslationPage />} />
              </Routes>
            </td>
          </tr>
        </tbody>
      </table>
    </div>
  );
}

export default App;
```

Main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
)
```

Index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/ong" href="/logo.png" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Translator-Inator</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Logo.png



Outputs

GlossaryManagementService

POST

The screenshot displays a REST client interface with a dark theme. At the top, the URL bar shows `http://localhost:5276/api/GlossaryManagement`. Below it, the method is set to **POST** and the same URL is entered. A **Send** button is visible on the right. The interface has tabs for Params, Authorization, Headers (9), Body, Pre-request Script, Tests, and Settings. The **Body** tab is active, showing a JSON payload in raw format:

```
1 {
2   "id": 0,
3   "word": "Hello",
4   "meaning": "A form of greting.",
5   "notes": "Often used to start a conversation."
6 }
```

 Below the request, the **Body** tab of the response is selected, showing a status of **201 Created** with a response time of **256 ms** and a size of **309 B**. The response body is displayed in pretty JSON format:

```
1 {
2   "id": 6,
3   "word": "Hello",
4   "meaning": "A form of greting.",
5   "notes": "Often used to start a conversation."
6 }
```


GET

 **http://localhost:5276/api/GlossaryManagement**  Save 

GET 

http://localhost:5276/api/GlossaryManagement

Send 



Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary

This request does not have a body

Body Cookies Headers (4) Test Results  200 OK 12 ms 250 B  Save Response 

Pretty Raw Preview Visualize JSON  

```
1  [
2    {
3      "id": 6,
4      "word": "Hello",
5      "meaning": "A form of greting.",
6      "notes": "Often used to start a conversation."
7    }
8  ]
```

PUT

 **http://localhost:5276/api/GlossaryManagement/6**

PUT

http://localhost:5276/api/GlossaryManagement/6

Send

Params

Authorization

Headers (9)

Body

Pre-request Script

Tests

Settings

none

form-data

x-www-form-urlencoded

raw

binary

JSON

1

2

3

4

5

6

"id": 6,

"word": "Hello",

"meaning": "A form of greting, mostly formal.",

"notes": "Often used to start a conversation."

Body

Cookies

Headers (2)

Test Results

204 No Content

105 ms

81 B

Save Response

Pretty

Raw

Preview

Visualize

Text

1

 **http://localhost:5276/api/GlossaryManagement**

GET

http://localhost:5276/api/GlossaryManagement

Send

Params

Authorization

Headers (7)

Body

Pre-request Script

Tests

Settings

Query Params

	Key	Value	Bulk Edit
	Key	Value	

Body

Cookies

Headers (4)

Test Results

200 OK

12 ms

265 B

Save Response

Pretty

Raw

Preview

Visualize

JSON

1

2

3

4

5

6

7

8

{

"id": 6,

"word": "Hello",

"meaning": "A form of greting, mostly formal.",

"notes": "Often used to start a conversation."

}

DELETE

 http://localhost:5276/api/GlossaryManagement

 Save



DELETE

▼

http://localhost:5276/api/GlossaryManagement/6

Send

▼



Params

Authorization

Headers (7)

Body

Pre-request Script

Tests

Settings

Cookies

Query Params

	Key	Value	Bulk Edit
	Key	Value	

Body

Cookies

Headers (2)

Test Results

 204 No Content 28 ms 81 B

Save Response

▼

Pretty

Raw

Preview

Visualize

Text

▼



1

 http://localhost:5276/api/GlossaryManagement

 Save



GET

▼

http://localhost:5276/api/GlossaryManagement

Send

▼



Params

Authorization

Headers (7)

Body

Pre-request Script

Tests

Settings

Cookies

Query Params

	Key	Value	Bulk Edit
	Key	Value	

Body

Cookies

Headers (4)

Test Results

 200 OK 8 ms 150 B

Save Response

▼

Pretty

Raw

Preview

Visualize

JSON

▼



1 

TranslationEngineService

POST

The screenshot displays a REST client interface with the following details:

- URL:** `http://localhost:5233/api/TranslationEngine`
- Method:** `POST`
- Body (Request):**

```
1  {"id": 0,  
2    "translationString": "Hello word!",  
3    "fromLang": "English",  
4    "toLang": "Tamil"  
5  }
```
- Response:**
 - Status:** 201 Created
 - Time:** 270 ms
 - Size:** 288 B
 - Body (Response):**

```
1  {"id": 1,  
2    "translationString": "Hello word!",  
3    "fromLang": "English",  
4    "toLang": "Tamil"  
5  }
```

GET

 **http://localhost:5233/api/TranslationEngine**  Save 

GET 

http://localhost:5233/api/TranslationEngine

Send 



Params Authorization Headers (7) **Body** Pre-request Script Tests Settings Cookies

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary

This request does not have a body

Body Cookies Headers (4) Test Results

 200 OK 11 ms 230 B [Save Response](#) 

Pretty Raw Preview Visualize

JSON 

```
1  [
2    {
3      "id": 1,
4      "translationString": "Hello word!",
5      "fromLang": "English",
6      "toLang": "Tamil"
7    }
8  ]
```

PUT

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:5233/api/TranslationEngine/1`
- Method:** `PUT`
- Body:** A JSON object:

```
{  "id": 0,  "translationString": "Hello there buddy!",  "fromLang": "English",  "toLang": "Tamil"}
```
- Response:** `204 No Content`, `105 ms`, `81 B`

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:5233/api/TranslationEngine`
- Method:** `GET`
- Response:** `200 OK`, `7 ms`, `237 B`
- Body:** A JSON object:

```
{  "id": 1,  "translationString": "Hello there buddy!",  "fromLang": "English",  "toLang": "Tamil"}
```

DELETE

 http://localhost:5233/api/TranslationEngine/1

 Save



DELETE

http://localhost:5233/api/TranslationEngine/1

Send



Params

Authorization

Headers (9)

Body

Pre-request Script

Tests

Settings

Cookies

Query Params

	Key	Value	Bulk Edit
	Key	Value	

Body

Cookies

Headers (2)

Test Results

 204 No Content 29 ms 81 B

Save Response

Pretty

Raw

Preview

Visualize

Text





1

 http://localhost:5233/api/TranslationEngine

 Save



GET

http://localhost:5233/api/TranslationEngine

Send



Params

Authorization

Headers (7)

Body

Pre-request Script

Tests

Settings

Cookies

Query Params

	Key	Value	Bulk Edit
	Key	Value	

Body

Cookies

Headers (4)

Test Results

 200 OK 10 ms 150 B

Save Response

Pretty

Raw

Preview

Visualize

JSON





1

LanguageDetectionService

POST

 http://localhost:5194/API/LanguageDetection

POST

http://localhost:5194/API/LanguageDetection

Send

Params

Authorization

Headers (9)

Body

Pre-request Script

Tests

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

JSON

Beautify

1

2

3

4

5

6

```
"id" : 0,
"name" : "Tamil",
"origin" : "India",
"notes" : ""
```

Body

Cookies

Headers (5)

Test Results

201 Created

240 ms

259 B

Save Response

Pretty

Raw

Preview

Visualize

JSON

1

2

3

4

5

6

```
"id": 1,
"name": "Tamil",
"origin": "India",
"notes": ""
```


GET

 **http://localhost:5194/API/LanguageDetection**

GET

http://localhost:5194/API/LanguageDetection

Send

Params

Authorization

Headers (7)

Body

Pre-request Script

Tests

Settings

Cookies

☒ none

☐ form-data

☐ x-www-form-urlencoded

☐ raw

☐ binary

This request does not have a body

Body

Cookies

Headers (4)

Test Results

200 OK 9 ms 201 B

Save Response

Pretty

Raw

Preview

Visualize

JSON

```
1 {
2   {
3     "id": 1,
4     "name": "Tamil",
5     "origin": "India",
6     "notes": ""
7   }
8 }
```

PUT

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:5194/API/LanguageDetection/1`
- Method:** `PUT`
- Body:** A JSON object: `{ "id" : 0, "name" : "Tamil", "origin" : "India", "notes" : "Specifically from the south." }`
- Response:** `204 No Content`, `100 ms`, `81 B`

The interface includes tabs for Params, Authorization, Headers (9), Body, Pre-request Script, Tests, and Settings. The Body tab is active, showing the JSON payload. The response section shows the status code, time, and size.

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:5194/API/LanguageDetection`
- Method:** `GET`
- Response:** `200 OK`, `21 ms`, `229 B`

The interface includes tabs for Params, Authorization, Headers (7), Body, Pre-request Script, Tests, and Settings. The Params tab is active, showing a table for Query Params:

Key	Value	Bulk Edit
Key	Value	

The Body tab is also visible, showing a JSON response: `{ "id": 1, "name": "Tamil", "origin": "India", "notes": "Specifically from the south." }`

DELETE

HTTP

http://localhost:5194/API/LanguageDetection

Save

</>

DELETE

http://localhost:5194/API/LanguageDetection/1

Send

Params

Authorization

Headers (7)

Body

Pre-request Script

Tests

Settings

Cookies

Query Params

	Key	Value	Bulk Edit
	Key	Value	

Body

Cookies

Headers (2)

Test Results

204 No Content

25 ms

81 B

Save Response

Pretty

Raw

Preview

Visualize

Text

1

HTTP

http://localhost:5194/API/LanguageDetection

Save

</>

GET

http://localhost:5194/API/LanguageDetection

Send

Params

Authorization

Headers (7)

Body

Pre-request Script

Tests

Settings

Cookies

Query Params

	Key	Value	Bulk Edit
	Key	Value	

Body

Cookies

Headers (4)

Test Results

200 OK

9 ms

150 B

Save Response

Pretty

Raw

Preview

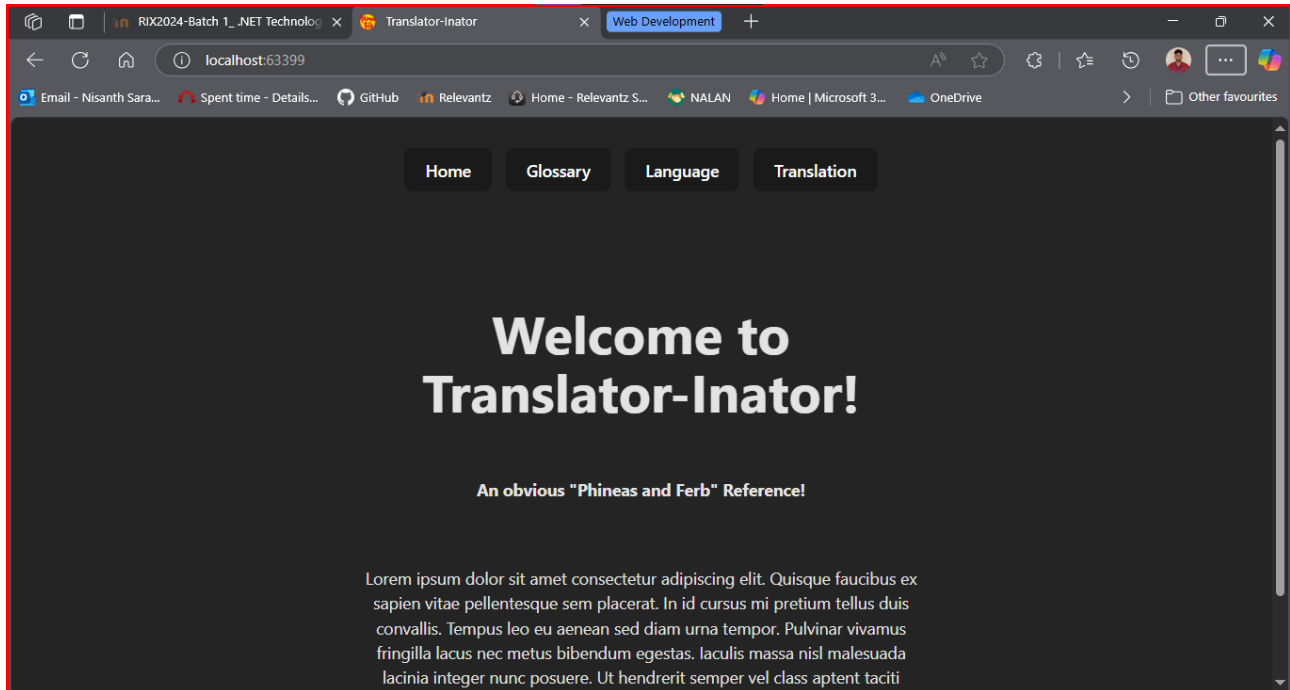
Visualize

JSON

1

Front-End Outputs that utilizes the gateway

Home Page



Glossary Manager

Glossary Manager

Enter the word.

Enter the meaning.

Any additional notes?

Add Glossary

No glossaries found.

Glossary Manager

Enter the word.

Please fill in this field.

Any additional notes?

Add Glossary

Glossary Manager

Hello

Greeting

Any additional notes?

Add Glossary

No glossaries found.

After adding...

Glossary Search:

Search

Glossary List

Word	Meaning	Additional Notes	Edit	Delete
Hello	Greeting	No Notes.	Edit	Delete

With no notes provided, the “No Notes.” is inserted instead.

Editing

HomeGlossaryLanguageTranslation

Glossary Manager

Hello

Greeting

No Notes.

Update Glossary

Glossary Manager

Hello

Greeting

Mostly Formal.

Update Glossary

Glossary List

Word	Meaning	Additional Notes	Edit	Delete
Hello	Greeting	Mostly Formal.	Edit	Delete

Searching

Glossary Search:

Hello

Search

localhost:63399 says

Search Results:
Word: Hello
Meaning: Greeting
Additional Notes: Mostly Formal.

OK

Glossary Search:

hi

Search

localhost:63399 says

No matching glossaries found.

OK

Deletion

Glossary List					
Word	Meaning	Additional Notes	Edit	Delete	
Hello	Greeting	Mostly Formal.	Edit	Delete	

Language Manager

The screenshot shows a web browser window with the URL `localhost:63399/languageManager`. The page has a dark theme and a navigation bar with four buttons: **Home**, **Glossary**, **Language** (which is highlighted), and **Translation**. Below the navigation bar, the title **Language Manager** is displayed. There are three input fields: "Enter the name.", "Enter the origin.", and "Any additional notes?". Below these fields is an **Add Language** button. At the bottom of the form, the text "No languages found." is displayed.

This screenshot shows the same Language Manager interface, but with the text "English" entered into the "Enter the name." input field. The other fields and the "Add Language" button remain visible. The text "No languages found." is still at the bottom.

This screenshot shows the "Language Search:" section and the "Language List" section. The search section has an input field with the placeholder "Enter the language name." and a **Search** button. Below this is the "Language List" section, which contains a table with the following data:

Name	Origin	Additional Notes	Edit	Delete
English	Not Given.	No Notes.	Edit	Delete

Meaning Origin and Notes can be null.

Editing

Language Manager

English

American?

No Notes.

Update Language

Language Search:

Enter the language name.

Search

Language List

Name	Origin	Additional Notes	Edit	Delete
English	American?	No Notes.	Edit	Delete

Searching

Email - Nisanth Sara...Spent time - Details...GitHub

localhost:63399 says

Search Results:
Name: English
Origin: American?
Additional Notes: No Notes.

OK

Language Search:

English

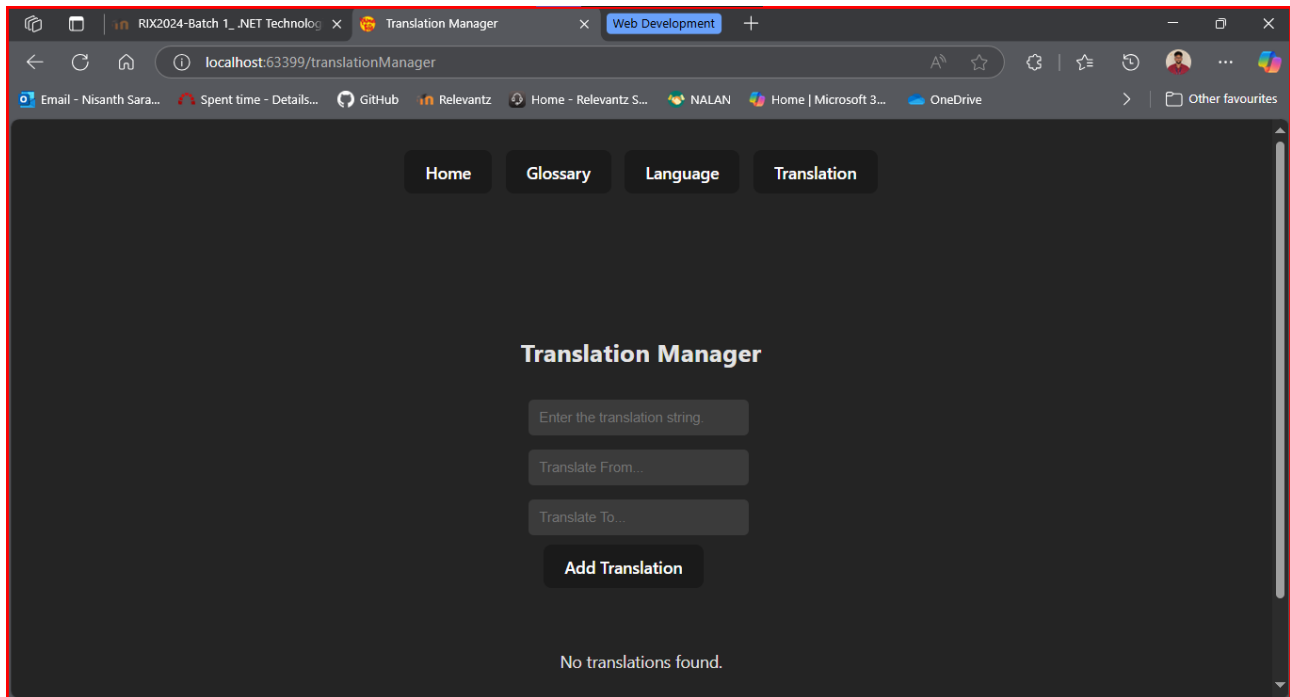
Search

Deleting

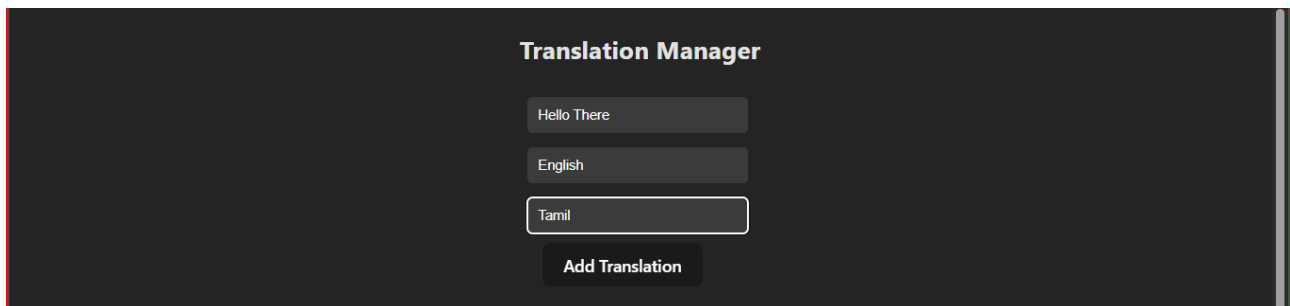
Language List

Name	Origin	Additional Notes	Edit	Delete
English	American?	No Notes.	Edit	Delete

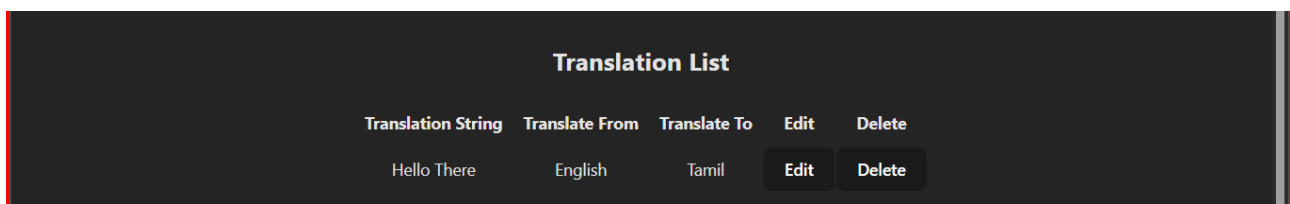
Translation Manager



The screenshot shows a web browser window with the URL `localhost:63399/translationManager`. The application has a dark theme and a navigation bar with buttons for Home, Glossary, Language, and Translation. The main heading is "Translation Manager". Below it, there are three input fields: "Enter the translation string.", "Translate From...", and "Translate To...". An "Add Translation" button is positioned below these fields. At the bottom, a message states "No translations found."

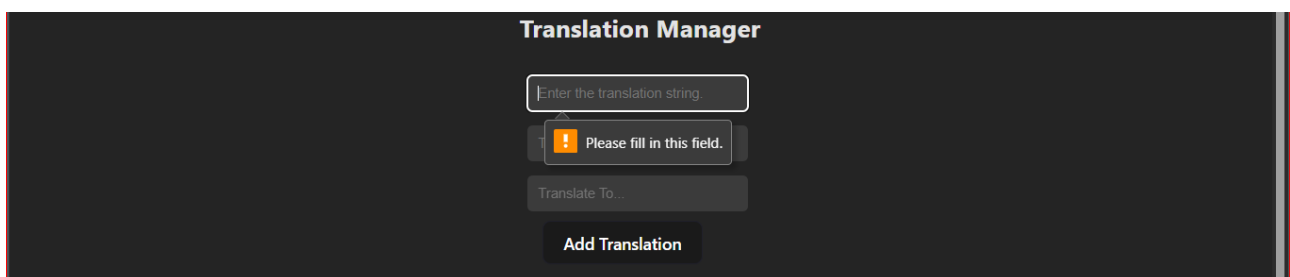


The screenshot shows the same application with the form fields populated. The "Enter the translation string." field contains "Hello There", the "Translate From..." field contains "English", and the "Translate To..." field contains "Tamil". The "Add Translation" button remains visible.



The screenshot shows a table titled "Translation List" with five columns: Translation String, Translate From, Translate To, Edit, and Delete. The table contains one row with the values "Hello There", "English", "Tamil", "Edit", and "Delete".

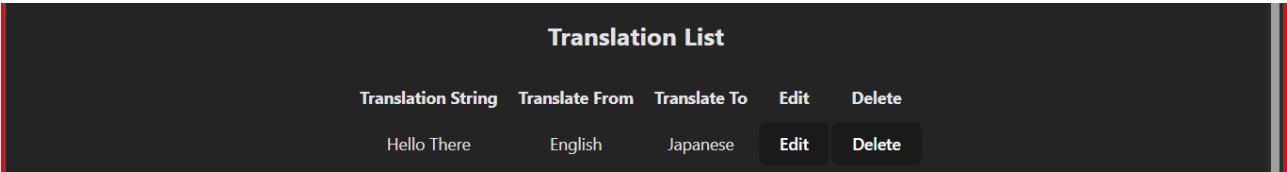
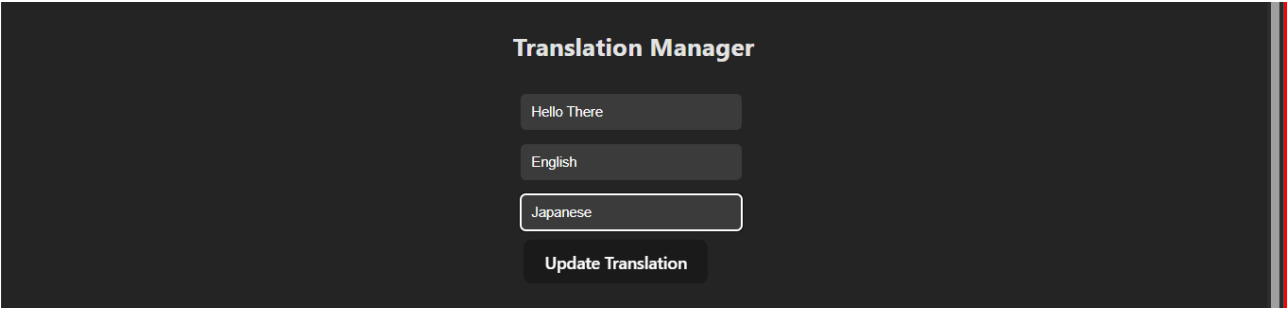
Translation String	Translate From	Translate To	Edit	Delete
Hello There	English	Tamil	Edit	Delete



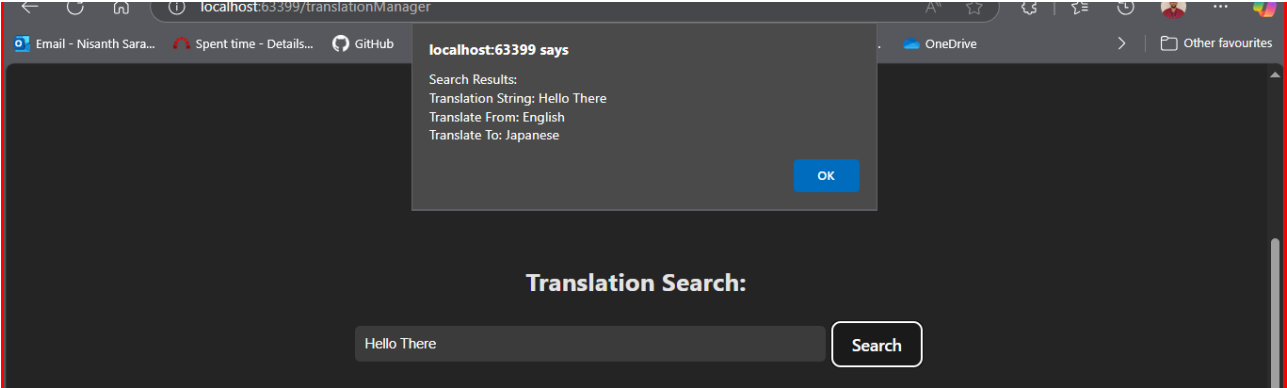
The screenshot shows the application with a validation error. The "Enter the translation string." field is empty, and a red error message "Please fill in this field." is displayed below it. The "Translate To..." field is also empty. The "Add Translation" button is still present.

All the fields are required.

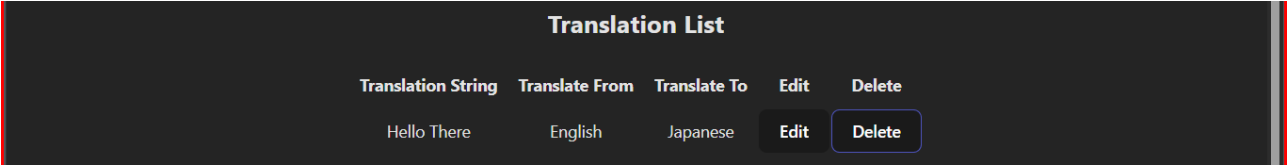
Editing



Searching



Deleting



FIN
